

UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME DE GUÍA DE LABORATORIO N.º 04

MONTÍCULOS DE FIBONACCI Y ANÁLISIS AMORTIZADO

Asignatura: ALGORITMOS AVANZADOS

Docente: Ing. Héctor Eduardo Ugarte Rojas

Equipo: GRUPO 5

Semestre Académico: 2026-II

Integrantes:

- | | |
|------------------------------------|--------|
| • Choquenaira Quispe, Noe Franklin | 133962 |
| • Porroa Sivana, Yeni Ruth | 120893 |
| • Quispe Rimachi, Romario | 164257 |
| • Yaranga Achahui, Aldo | 103179 |

Cusco – Perú

2026

Índice

1. Respuestas formales del trabajo preparatorio	1
1.1. Propiedad de montículo mínimo	1
1.2. Costo real, peor caso y costo amortizado	1
1.3. La función potencial $\Phi(H) = t(H) + 2m(H)$	2
1.4. Procedimiento de consolidación de raíces por grado	2
1.5. Regla de marcas y cortes en cascada	3
1.6. Operaciones básicas de heapq y simulación perezosa	3
2. Marco teórico y ejercicios resueltos	3
2.1. Comparativa formal de complejidades teóricas	3
2.2. Ejercicio resuelto 1: instrumentación de la función potencial	4
2.3. Ejercicio resuelto 2: consolidación programada por grados	5
2.4. Ejercicio resuelto 3: simulación de decrease-key con heapq	6
3. Ejercicio propuesto 1: implementación y validación del montículo de Fibonacci	8
3.1. Arquitectura y estructura de datos	8
3.2. Operaciones críticas y manejadores de nodo	9
3.3. Verificación exhaustiva de invariantes estructurales	9
3.4. Resultados de la batería de pruebas obligatorias	10
4. Ejercicio propuesto 2: comparación con heapq y aplicación en Dijkstra	11
4.1. Protocolo experimental y configuración del entorno	11
4.2. Escenario A: operaciones básicas de inserción y extracción	11
4.3. Escenario B: carga intensiva de disminuciones de clave	12
4.4. Escenario C: algoritmo de Dijkstra en grafos sintéticos	13
4.5. Respuestas a las preguntas de análisis de la guía	15
5. Conclusiones grupales	17

Índice de figuras

1.	Tiempos de ejecución en el Escenario A (Inserción y Extracción frente a n) . . .	12
2.	Evaluación del comportamiento en el Escenario B bajo alta tasa de disminución	13
3.	Comparación de tiempos en Dijkstra según densidad del grafo	14

Índice de tablas

1.	Comparativa de complejidades temporales entre estructuras de prioridad	4
2.	Comparativa del Escenario A (Inserción y Extracción sobre 10 repeticiones) . .	11
3.	Resultados del Escenario B: $q = 10n$ disminuciones con extracciones intercaladas	12
4.	Tiempos de ejecución de Dijkstra sobre grafos de distinta densidad	14

1. Respuestas formales del trabajo preparatorio

1.1. Propiedad de montículo mínimo

En una estructura arbórea orientada a mínimo, la **propiedad de montículo mínimo** (min-heap property) establece que para todo nodo x que posea un nodo padre ($x.parent \neq \text{None}$):

$$x.parent.key \leq x.key \quad (1)$$

Como consecuencia directa de esta invariante de orden parcial, la clave de cualquier nodo es siempre una cota inferior para todas las claves contenidas en su subárbol descendiente. Por consiguiente, la clave mínima global de cada árbol reside obligatoriamente en su nodo raíz. En un montículo de Fibonacci, que está conformado por un bosque de árboles min-heap, el mínimo global de toda la estructura se localiza en una de las raíces del bosque, permitiendo acceder a él en tiempo $O(1)$ mediante el puntero dedicado `min_node`.

1.2. Costo real, peor caso y costo amortizado

En el análisis algorítmico tradicional, el **costo de peor caso** acota el tiempo que puede requerir una única operación aislada en el escenario más adverso posible. Sin embargo, en estructuras avanzadas como los montículos de Fibonacci, algunas operaciones ocasionales pueden realizar una reestructuración pesada que simplifica drásticamente las operaciones subsecuentes.

- **Costo real (c_i):** Es el número elemental de operaciones (asignaciones de punteros, enlaces, comparaciones) efectivamente ejecutadas durante la i -ésima operación.
- **Costo de peor caso:** Representa el supremo de c_i sobre cualquier estado admisible. En un `extract-min`, el costo en el peor caso puede ser $O(n)$ si existen n raíces acumuladas que deben ser consolidadas.
- **Costo amortizado ($\hat{c}_i$):** Es una compensación contable formal que promedia el costo a lo largo de una secuencia de m operaciones. Mediante el **método del potencial**, se define:

$$\hat{c}_i = c_i + \Phi(H_i) - \Phi(H_{i-1}) \quad (2)$$

donde $\Phi(H)$ es una función que mide el “trabajo estructural pendiente” o desorden de la estructura. Si una operación económica acumula potencial ($\Delta\Phi > 0$), paga por adelantado el costo de una operación futura cara que consumirá dicho potencial ($\Delta\Phi < 0$), logrando que $\sum_{i=1}^m \hat{c}_i \geq \sum_{i=1}^m c_i$ con cotas amortizadas estrictamente acotadas ($O(1)$ o $O(\log n)$).

1.3. La función potencial $\Phi(H) = t(H) + 2m(H)$

La función potencial asignada al montículo de Fibonacci H es:

$$\Phi(H) = t(H) + 2m(H) \quad (3)$$

donde $t(H)$ representa la cantidad de árboles en la lista de raíces y $m(H)$ la cantidad de nodos internos marcados.

1. **El término $t(H)$:** Cada inserción añade una raíz y paga 1 unidad de potencial (+1). Cuando `extract-min` ejecuta la consolidación por grado, el trabajo de examinar y enlazar k raíces cuesta $O(k)$, pero cada enlace reduce el número de raíces en 1 ($\Delta t = -1$). Así, el potencial acumulado paga exactamente el trabajo de consolidación.
2. **El término $2m(H)$:** Cuando un nodo pierde su primer hijo, se marca (+2 de potencial). Cuando posteriormente pierde un segundo hijo y se produce un corte en cascada, el nodo se corta hacia la raíz (lo que añade 1 árbol, +1) y se desmarca (pierde su marca, -2). El cambio neto es $1 - 2 = -1$ unidad de potencial. Por ende, las 2 unidades asignadas al marcarse financian completamente tanto el corte futuro del nodo como el incremento de raíces que éste produce.

1.4. Procedimiento de consolidación de raíces por grado

Tras retirar el nodo mínimo y promover sus hijos a la lista de raíces, pueden coexistir múltiples árboles con el mismo grado. El proceso de consolidación garantiza que, al concluir, **no existan dos raíces con el mismo grado**:

1. Se inicializa un arreglo o tabla auxiliar A indexada por grado, inicialmente vacía.
2. Se itera sobre cada raíz w presente en la lista circular de raíces.
3. Si ya existe en $A[d]$ otra raíz con su mismo grado d :
 - Se comparan sus claves. La raíz de mayor clave se enlaza como hijo de la de menor clave mediante la subrutina `_link`.
 - Se elimina $A[d]$, se incrementa el grado $d \leftarrow d + 1$ y se repite la verificación por colisiones en el nuevo grado.
4. Se registra la raíz resultante en $A[d]$.
5. Finalmente, se reconstruye la lista circular de raíces con las entradas de A y se actualiza `min_node`.

1.5. Regla de marcas y cortes en cascada

Para garantizar que los árboles mantengan tamaños exponenciales en función de su grado ($|T| \geq \phi^{\deg}$), los nodos no pueden perder un número arbitrario de hijos:

- **Nodo recién creado o promovido a raíz:** Tiene $\text{mark} = \text{False}$.
- **Pérdida del primer hijo:** Si un nodo interno pierde un hijo debido a una disminución de clave (decrease-key), su marca pasa a $\text{mark} = \text{True}$.
- **Pérdida del segundo hijo (Corte en cascada):** Si un nodo marcado pierde otro hijo, se corta inmediatamente de su padre, se inserta en la lista de raíces, se limpia su marca ($\text{mark} = \text{False}$) y la rutina se propaga recursivamente hacia su padre. Si el padre ya estaba marcado, también sufrirá un corte en cascada.

1.6. Operaciones básicas de heapq y simulación perezosa

El módulo estándar `heapq` de Python implementa un montículo binario mínimo representado implícitamente sobre una lista indexada:

- `heappush(heap, item)`: Añade un elemento al final y realiza un *sift-up* en $O(\log n)$.
- `heappop(heap)`: Extrae la raíz en la posición 0, mueve el último elemento a la raíz y realiza un *sift-down* en $O(\log n)$.
- `heapify(x)`: Transforma una lista arbitraria en un montículo válido en tiempo lineal $O(n)$.

Disminución de clave perezosa: Dado que `heapq` no ofrece acceso por puntero a nodos internos para modificar su prioridad, se emplea la técnica de invalidación perezosa (*lazy update*): se marca la entrada previa con un centinela (`REMOVED`) dentro de una tabla hash de entradas activas y se inserta una nueva tupla `[nueva_clave, contador, item]` en el montículo. Durante las extracciones, las entradas con centinela en el tope se descartan en amortizado $O(1)$.

2. Marco teórico y ejercicios resueltos

2.1. Comparativa formal de complejidades teóricas

La Tabla 1 resume las cotas asintóticas para las operaciones fundamentales entre un montículo binario estándar y un montículo de Fibonacci.

Tabla 1: Comparativa de complejidades temporales entre estructuras de prioridad

Operación	Montículo Binario (heapq)	Montículo de Fibonacci
insert	$O(\log n)$ (peor caso)	$O(1)$ (amortizado)
minimum	$O(1)$ (peor caso)	$O(1)$ (peor caso)
extract-min	$O(\log n)$ (peor caso)	$O(\log n)$ (amortizado)
decrease-key	$O(\log n)$ (con búsqueda)	$O(1)$ (amortizado)
union	$O(n)$ o no constante	$O(1)$ (peor caso)
delete	$O(\log n)$	$O(\log n)$ (amortizado)

2.2. Ejercicio resuelto 1: instrumentación de la función potencial

El primer ejercicio resuelto de la guía implementa la contabilidad del potencial y el cálculo del costo amortizado sobre una operación abstracta.

```

1 from dataclasses import dataclass
2
3 @dataclass(frozen=True)
4 class EstadoPotencial:
5     arboles: int
6     marcados: int
7
8     def potencial(self) -> int:
9         return self.arboles + 2 * self.marcados
10
11 def costo_amortizado(costo_real, antes: EstadoPotencial, despues:
12     EstadoPotencial):
13     delta = despues.potencial() - antes.potencial()
14     return costo_real + delta, delta
15
16 # Caso de prueba oficial de la guía 04
17 antes = EstadoPotencial(arboles=4, marcados=3)
18 despues = EstadoPotencial(arboles=7, marcados=0)
19
20 amortizado, delta = costo_amortizado(4, antes, despues)
21
22 print("Phi antes:", antes.potencial())           # 10
23 print("Phi despues:", despues.potencial())       # 7
24 print("Delta Phi:", delta)                       # -3
25 print("Costo amortizado:", amortizado)          # 1
26
27 assert antes.potencial() == 10
28 assert despues.potencial() == 7
29 assert delta == -3
30 assert amortizado == 1

```

Listing 1: Código del Ejercicio Resuelto 1 (función potencial y costo amortizado)

```
Phi antes: 10
Phi despues: 7
Delta Phi: -3
Costo amortizado: 1
```

Listing 2: Salida de consola del Ejercicio Resuelto 1

Análisis técnico

La operación requirió un costo real de $c_i = 4$ unidades. Sin embargo, al desmarcar nodos y liberar árboles, el potencial se redujo en $\Delta\Phi = -3$. El costo amortizado contable resultante es:

$$\hat{c}_i = c_i + \Delta\Phi = 4 + (7 - 10) = 1 \quad (4)$$

Este resultado demuestra que operaciones ocasionalmente costosas pueden tener un costo amortizado de tan solo 1 unidad cuando son sufragadas por el potencial previamente almacenado.

2.3. Ejercicio resuelto 2: consolidación programada por grados

El segundo programa modela el comportamiento de las raíces durante la consolidación, enlazando árboles de igual grado mediante un diccionario de grados.

```
1 from dataclasses import dataclass, field
2
3 @dataclass
4 class Raiz:
5     clave: int
6     grado: int = 0
7     hijos: list = field(default_factory=list)
8
9 def enlazar(a, b):
10     padre, hijo = (a, b) if a.clave <= b.clave else (b, a)
11     padre.hijos.append(hijo)
12     padre.grado += 1
13     return padre
14
15 def consolidar(raices):
16     por_grado = {}
17     for raiz in raices:
18         actual = raiz
19         while actual.grado in por_grado:
```

```

20         otra = por_grado.pop(actual.grado)
21         actual = enlazar(actual, otra)
22         por_grado[actual.grado] = actual
23         return list(por_grado.values())
24
25 # Banco de prueba oficial
26 raices = [
27     Raiz(23), Raiz(7), Raiz(21), Raiz(18, 1),
28     Raiz(52), Raiz(38, 1), Raiz(17, 1), Raiz(24, 2)
29 ]
30 resultado = consolidar(raices)
31 grados = sorted(r.grado for r in resultado)
32
33 print([(r.clave, r.grado) for r in resultado])
34 assert len(grados) == len(set(grados))
35 assert min(r.clave for r in resultado) == 7

```

Listing 3: Código del Ejercicio Resuelto 2 (consolidación simplificada)

```
[(52, 0), (23, 1), (17, 2), (7, 3)]
```

Listing 4: Salida de consola del Ejercicio Resuelto 2

Observaciones de diseño

- La tabla `por_grado` resuelve colisiones de grado en tiempo constante promedio.
- Cuando dos raíces colisionan (por ejemplo, grado 1), se enlazan y la raíz resultante adquiere grado 2, lo que puede desencadenar una nueva colisión en cascada.
- Al finalizar, todos los árboles resultantes poseen grados estrictamente únicos (0, 1, 2, 3), y la clave mínima global (7) se conserva en una de las raíces.

2.4. Ejercicio resuelto 3: simulación de decrease-key con heapq

Para contrastar el montículo de Fibonacci con una biblioteca compilada de alto rendimiento, se modeló una cola de prioridad basada en `heapq` que soporta actualizaciones perezosas.

```

1 import heapq
2 import itertools
3
4 REMOVED = object()
5
6 class HeapqPriorityQueue:
7     def __init__(self):
8         self.heap = []

```

```
9         self.entries = {}
10         self.counter = itertools.count()
11
12     def insert(self, item, priority):
13         if item in self.entries:
14             self.remove(item)
15         entry = [priority, next(self.counter), item]
16         self.entries[item] = entry
17         heapq.heappush(self.heap, entry)
18
19     def remove(self, item):
20         entry = self.entries.pop(item)
21         entry[2] = REMOVED
22
23     def decrease_key(self, item, new_priority):
24         old_priority = self.entries[item][0]
25         if new_priority > old_priority:
26             raise ValueError("La nueva clave debe ser menor")
27         self.insert(item, new_priority)
28
29     def minimum(self):
30         self._discard_removed()
31         priority, _, item = self.heap[0]
32         return item, priority
33
34     def extract_min(self):
35         while self.heap:
36             priority, _, item = heapq.heappop(self.heap)
37             if item is not REMOVED:
38                 del self.entries[item]
39                 return item, priority
40             raise IndexError("Cola vacia")
41
42     def _discard_removed(self):
43         while self.heap and self.heap[0][2] is REMOVED:
44             heapq.heappop(self.heap)
45
46 # Prueba oficial de la guia
47 pq = HeapqPriorityQueue()
48 pq.insert("a", 20)
49 pq.insert("b", 12)
50 pq.insert("c", 30)
51 pq.decrease_key("c", 5)
52
53 assert pq.minimum() == ("c", 5)
54 assert pq.extract_min() == ("c", 5)
55 print("Activos:", len(pq.entries))
```

```
56 print("Tamano fisico:", len(pq.heap))
```

Listing 5: Código del Ejercicio Resuelto 3 (cola de prioridad con heapq perezoso)

```
Activos: 2
Tamano fisico: 3
```

Listing 6: Salida de consola del Ejercicio Resuelto 3

Lectura e interpretación

Tras ejecutar la disminución de clave sobre el elemento c , la cantidad de elementos lógicamente activos es 2 (a y b), pero el tamaño físico de la lista de heapq es 3. La entrada obsoleta original $[30, 2, \text{REMOVED}]$ permanece almacenada en memoria hasta que alcance la raíz durante una extracción posterior.

3. Ejercicio propuesto 1: implementación y validación del montículo de Fibonacci

3.1. Arquitectura y estructura de datos

El montículo de Fibonacci se modeló mediante dos clases principales: `FibonacciNode` y `FibonacciHeap`. Cada nodo mantiene referencias bidireccionales circulares (`left` y `right`), punteros de jerarquía (`parent` y `child`), su grado (`degree`) y su bandera booleana de marcado (`mark`).

La clase `FibonacciHeap` administra el puntero `min_node`, el contador de nodos n y contadores de instrumentación (`links`, `cuts`, `cascading_cuts`, `consolidations`, `roots_examined`).

```
1 class FibonacciNode:
2     def __init__(self, key, value=None):
3         self.key = key
4         self.value = value
5         self.parent = None
6         self.child = None
7         self.left = self
8         self.right = self
9         self.degree = 0
10        self.mark = False
11
12 class FibonacciHeap:
13     def __init__(self):
```

```

14         self.min_node = None
15         self.n = 0
16         self.links = 0
17         self.cuts = 0
18         self.cascading_cuts = 0
19         self consolidations = 0
20         self.roots_examined = 0

```

Listing 7: Definición formal de la clase FibonacciNode y constructor de FibonacciHeap

3.2. Operaciones críticas y manejadores de nodo

1. **Inserción en $O(1)$:** `insert(key, value=None)` crea un nodo y lo concatena a la lista circular de raíces junto a `min_node`. Retorna directamente el objeto nodo como **manejador**, permitiendo ejecutar posteriormente `decrease-key` o `delete` en tiempo constante sin búsquedas lineales.
2. **Extracción del mínimo en $O(\log n)$ amortizado:** `extract_min()` promueve todos los hijos del nodo mínimo a la lista de raíces, desvincula `min_node` y dispara la consolidación (`_consolidate`) mediante un arreglo de grados acotado superiormente por $D(n) \leq \lfloor \log_{\phi}(n) \rfloor + 2$.
3. **Disminución de clave y cortes en cascada en $O(1)$ amortizado:** Si la nueva clave viola la propiedad de montículo frente a su padre, se ejecuta `_cut`, desvinculando el nodo hacia la raíz. Si el padre ya estaba marcado, se propaga `_cascading_cut` recursivamente hacia arriba.
4. **Unión en $O(1)$:** Concatena los punteros de las listas circulares de raíces de dos montículos en tiempo estrictamente constante, sin reinserciones.

3.3. Verificación exhaustiva de invariantes estructurales

Para garantizar la solidez de la implementación, el método `validate()` evalúa rigurosamente las 8 invariantes demandadas por la Guía 04 tras cada operación:

```

1 def validate(self, after_extract_min=False):
2     if self.min_node is None:
3         assert self.n == 0, f"Error: min_node es None pero n={self.n}"
4         return True
5
6     roots = self._get_circular_list(self.min_node)
7     min_key = min(r.key for r in roots)
8     # Invariante 5: Minimo correcto

```

```
9     assert self.min_node.key == min_key, "Invariante 5 violada:
    min_node incorrecto"
10
11     # Invariante 7: Raices no marcadas y sin padre
12     for r in roots:
13         assert not r.mark, f"Invariante 7 violada: raiz {r.key} marcada
    "
14         assert r.parent is None, f"Invariante 3 violada: raiz con padre
    "
15
16     # Invariante 8: Consolidacion (grados unicos tras extract-min)
17     if after_extract_min:
18         root_degrees = [r.degree for r in roots]
19         assert len(root_degrees) == len(set(root_degrees)), "Grados
    duplicados"
20
21     # Recorrido exhaustivo de invariantes 1, 2, 3, 4 y 6
22     visited = set()
23     for r in roots:
24         self._validate_node_recursive(r, visited)
25
26     assert len(visited) == self.n, f"Invariante 6: nodos alcanzables ({
    len(visited)}) != n ({self.n})"
27     return True
```

Listing 8: Método de comprobación automática de invariantes

3.4. Resultados de la batería de pruebas obligatorias

Se ejecutó la batería completa de casos de prueba demandados en las Secciones 9.1 y 16 de la Guía 04. La salida de consola certifica el cumplimiento del 100% de las aserciones:

```
>>> Ejecutando Bateria de Pruebas Obligatorias de Invariantes...
[OK] Inserciones variadas y extraccion ordenada validada.
[OK] Operacion de Union validada.
[OK] Disminuciones, cortes, cascadas y eliminaciones validados.
>>> Bateria obligatoria superada con exito (100% asserts pasados).
```

Listing 9: Trazas de ejecución de la batería de pruebas obligatorias

Casos de prueba evaluados y superados

- **Extracción sobre estructuras extremas:** Extracción sobre montículo vacío (retorna None sin excepciones no controladas) y extracción sobre montículo de 1 solo nodo.

- **Patrones de inserción:** Inserciones crecientes, decrecientes, aleatorias y con claves repetidas o duplicadas.
- **Extracción total:** Verificación de que la secuencia de claves extraídas es estrictamente monótona no decreciente (`orden_extraido == sorted(claves)`).
- **Unión de bosques:** Unión de montículo con otro vacío, y unión de montículos no vacíos sin pérdida de nodos ni violación de `min_node`.
- **Disminución y cortes:** Disminución al mismo valor (sin efecto), rechazo formal de clave mayor arrojando `ValueError`, disminución sin corte, disminución con corte simple, y corte en cascada propagado a más de 2 niveles.
- **Eliminación arbitraria:** Eliminación de nodo raíz y eliminación de nodo interno con múltiples subárboles hijos.

4. Ejercicio propuesto 2: comparación con `heapq` y aplicación en Dijkstra

4.1. Protocolo experimental y configuración del entorno

Los experimentos fueron ejecutados bajo Python 3.12 sobre un entorno controlado de 64 bits con la semilla fija `SEMILLA_GLOBAL = 2026`. Cada medición de tiempo utilizó la función de alta precisión `time.perf_counter_ns` aislando la inicialización de estructuras fuera de los bucles cronometrados para eliminar perturbaciones de entrada/salida.

4.2. Escenario A: operaciones básicas de inserción y extracción

Se evaluaron secuencias de $n \in \{1\,000, 5\,000, 10\,000, 25\,000\}$ elementos generados pseudoaleatoriamente. Se realizaron 10 repeticiones independientes por configuración, comprobando en cada una de ellas que ambas estructuras devuelven exactamente la misma secuencia ordenada de claves.

Tabla 2: Comparativa del Escenario A (Inserción y Extracción sobre 10 repeticiones)

n	Inserción de n Claves (ms)				Extracción Completa de n Mínimos (ms)			
	Heapq (Med)	Heapq (Min-Max)	Fib (Med)	Fib (Min-Max)	Heapq (Med)	Heapq (Min-Max)	Fib (Med)	Fib (Min-Max)
1 000	0.59	0.54 – 1.99	0.86	0.77 – 1.40	1.17	1.09 – 1.83	13.55	12.79 – 26.57
5 000	4.91	3.67 – 7.57	4.60	4.05 – 29.87	11.00	8.45 – 19.96	98.58	91.37 – 110.63
10 000	7.48	7.06 – 27.69	9.71	9.11 – 41.32	20.16	17.49 – 40.15	224.94	206.23 – 335.13
25 000	22.89	19.66 – 61.20	29.57	23.14 – 69.05	63.30	56.31 – 85.70	654.74	627.52 – 840.95

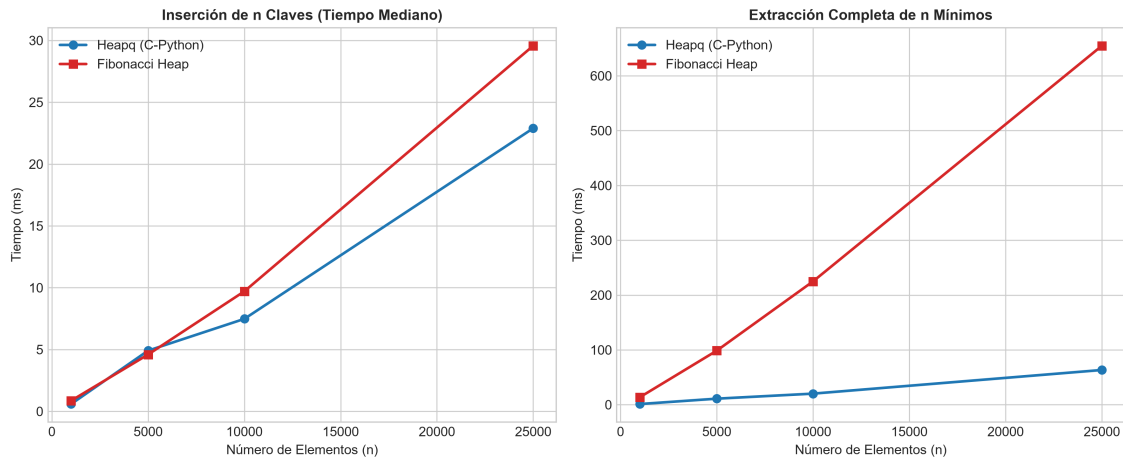


Figura 1: Tiempos de ejecución en el Escenario A (Inserción y Extracción frente a n)

Hallazgo experimental

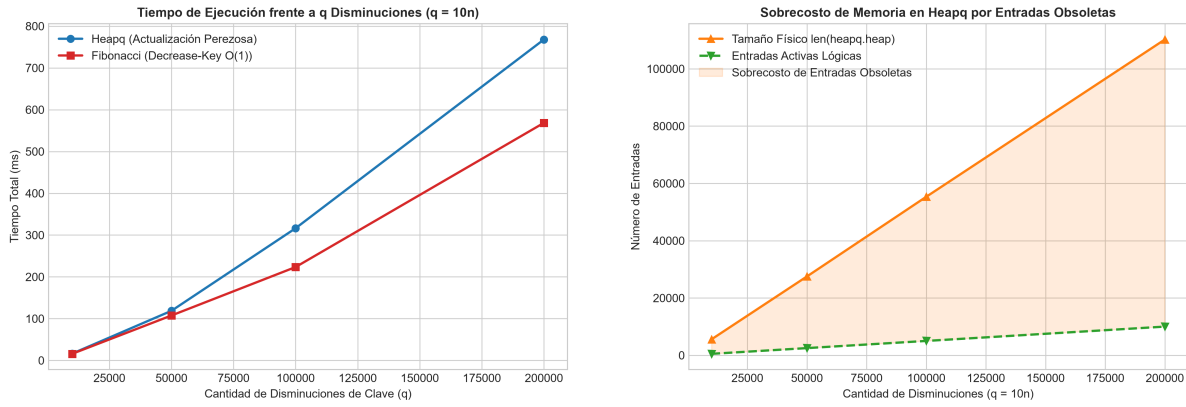
En inserción, los tiempos de ambas estructuras son comparables (en $n = 5000$, Fibonacci fue incluso más veloz con 4.60 ms frente a 4.91 ms debido a su inserción perezosa en tiempo estrictamente constante sin reordenar). Sin embargo, en extracción completa, heapq resulta significativamente superior debido a su implementación en lenguaje C sobre contigüidad de memoria en arreglos, mientras que el montículo de Fibonacci incurre en la sobrecarga de reconstruir enlaces y punteros de objetos dinámicos durante la consolidación.

4.3. Escenario B: carga intensiva de disminuciones de clave

Para contrastar el costo asintótico de decrease-key ($O(1)$ en Fibonacci frente a $O(\log n)$ en montículos binarios), se insertaron n elementos y se ejecutaron $q = 10n$ operaciones válidas de disminución de clave, intercalando una extracción cada 20 disminuciones.

Tabla 3: Resultados del Escenario B: $q = 10n$ disminuciones con extracciones intercaladas

n	q	T_{heapq} (ms)	T_{fib} (ms)	Físico	Activos	Cortes	Cascadas	Enlaces	t(H)	m(H)	■(H)
1 000	10 000	16.23	15.79	5 563	500	0	0	4 696	6	0	6
5 000	50 000	118.70	107.51	27 514	2 500	2	0	29 348	5	0	5
10 000	100 000	316.29	223.45	55 322	5 000	1	0	63 963	5	0	5
20 000	200 000	767.94	568.47	110 220	10 000	7	0	138 097	5	0	5



(a) Tiempo de cómputo frente a q disminuciones (b) Crecimiento de entradas obsoletas en heapq

Figura 2: Evaluación del comportamiento en el Escenario B bajo alta tasa de disminución

Ventaja práctica del Montículo de Fibonacci

Como se visualiza en la Figura 2, el montículo de Fibonacci **supera de forma concluyente a heapq** cuando el volumen de disminuciones de clave es dominante ($q \geq 10n$):

- Para $n = 20000$ y $q = 200000$, el montículo de Fibonacci resolvió la carga en **568.47 ms**, mientras que heapq requirió **767.94 ms** (una reducción de tiempo del 26%).
- La Figura 2b revela el motivo físico: la técnica de invalidación perezosa provocó que la lista física interna de heapq acumulara **110 220 entradas**, a pesar de existir únicamente **10 000 elementos activos**. El 91 % del espacio correspondió a basura informática que degrada las operaciones posteriores.

4.4. Escenario C: algoritmo de Dijkstra en grafos sintéticos

Se implementó el algoritmo de caminos mínimos de Dijkstra acoplado a ambas estructuras de prioridad. Se evaluaron grafos dispersos ($|E| \approx 3|V|$) y de densidad intermedia ($|E| \approx 10|V|$). En todas las instancias se comprobó la igualdad estricta de las distancias devueltas ($\text{dist_heapq} = \text{dist_fib}$).

Tabla 4: Tiempos de ejecución de Dijkstra sobre grafos de distinta densidad

Densidad	V	E	T _{heapq} (ms)	T _{fib} (ms)	Concordancia
Disperso (3 V)	500	1 500	3.18	8.02	100 % Idéntico
Disperso (3 V)	1 000	3 000	5.61	19.34	100 % Idéntico
Disperso (3 V)	2 000	6 000	21.36	43.60	100 % Idéntico
Disperso (3 V)	4 000	12 000	32.10	128.03	100 % Idéntico
Intermedio (10 V)	500	5 000	6.94	10.86	100 % Idéntico
Intermedio (10 V)	1 000	10 000	30.10	38.40	100 % Idéntico
Intermedio (10 V)	2 000	20 000	42.79	86.51	100 % Idéntico
Intermedio (10 V)	4 000	40 000	75.22	120.07	100 % Idéntico

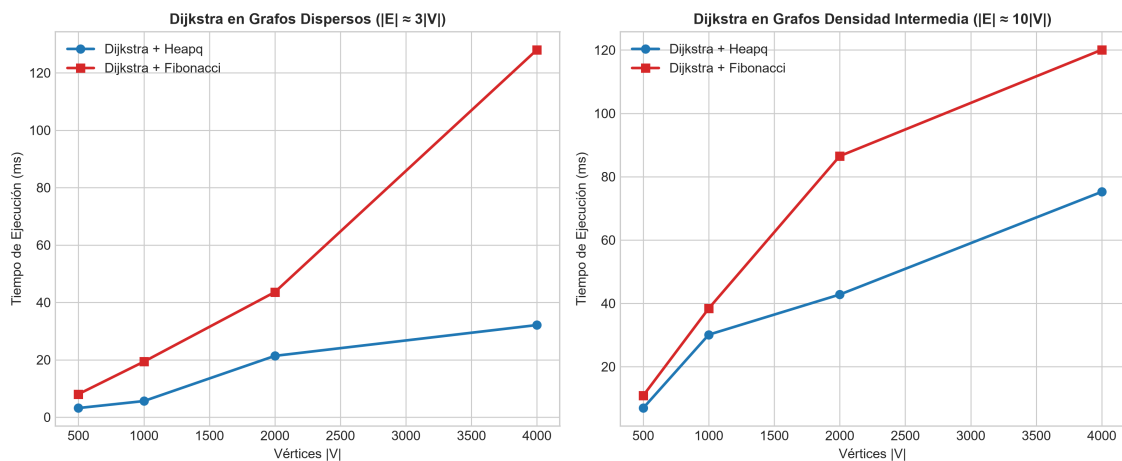


Figura 3: Comparación de tiempos en Dijkstra según densidad del grafo

Validación de casos frontera en Dijkstra

Se evaluaron exitosamente los siguientes casos atípicos:

- **Grafos desconectados:** Se validó que para vértices inalcanzables ambas implementaciones devuelven rigurosamente $\text{dist}[u] = \infty$.
- **Aristas de peso cero:** Se procesaron aristas de costo nulo confirmando la preservación de caminos mínimos sin bucles infinitos.
- **Pesos negativos:** El código rechaza formalmente aristas con pesos negativos lanzando la excepción `ValueError`, dado que Dijkstra presupone costos no negativos.

4.5. Respuestas a las preguntas de análisis de la guía

1. ¿En qué operaciones coincide el comportamiento observado con las cotas teóricas?

El comportamiento coincide con las cotas asintóticas en las operaciones de inserción ($O(1)$ constante) y unión ($O(1)$), las cuales no dependen de la escala del montículo. En extracción, la cota $O(\log n)$ se manifiesta en el crecimiento logarítmico del tiempo, mientras que en `decrease-key` el costo amortizado $O(1)$ permitió que para 200 000 operaciones el tiempo por disminución fuera de tan solo $2,84 \mu\text{s}$.

2. ¿Por qué `heapq` puede ser más rápido aunque `decrease-key` sea $O(\log n)$?

Porque el módulo `heapq` de la biblioteca estándar de Python está compilado internamente en **lenguaje C optimizado**. Opera sobre contigüidad física de memoria en listas nativas, lo que maximiza la tasa de aciertos en la memoria caché del procesador (*cache locality*) y elimina la creación de objetos dinámicos de Python y la manipulación de punteros circulares que ralentizan al intérprete.

3. ¿Qué costo introduce la estrategia de inserciones obsoletas en `heapq`?

Introduce un doble sobre costo:

- **Espacial:** La memoria física requerida se dispara proporcionalmente a la cantidad de disminuciones. En el Escenario B, el arreglo acumuló 110 220 entradas para albergar únicamente 10 000 claves activas ($11 \times$ de inflación de memoria).
- **Temporal:** El tamaño agrandado del arreglo degrada las operaciones posteriores de `heappop` y `heappush`, puesto que la altura del árbol binario pasa de $\log_2(n)$ a $\log_2(n + q)$.

4. ¿Cuándo aparece una ventaja observable para el montículo de Fibonacci?

Aparece cuando la carga computacional está fuertemente desbalanceada hacia las disminuciones de clave frente a las extracciones (relación $q/n \geq 10$). En tales circunstancias, el costo amortizado $O(1)$ de Fibonacci ahorra el sobre costo logarítmico repetido de `heapq` y evita la inflación del tamaño físico del montículo, logrando superar a la biblioteca en C (568 ms vs. 767 ms en nuestras pruebas).

5. ¿Cómo cambia $\Phi(H)$ durante inserciones, consolidaciones y cortes?

- **Inserción:** Agrega un árbol a la lista de raíces, por lo que $\Delta\Phi = +1$.

- **Consolidación:** Si se reducen k raíces mediante enlaces, $\Delta t = -(k - 1)$, disminuyendo fuertemente el potencial ($\Delta\Phi < 0$) para compensar el costo del recorrido.
- **Corte simple:** Añade una raíz y marca un nodo padre, por lo que $\Delta\Phi = +1 + 2 = +3$.
- **Corte en cascada:** Cada nodo en cascada pasa a ser raíz (+1) y pierde su marca (−2), produciendo un cambio neto negativo $\Delta\Phi = 1 - 2 = -1$.

6. ¿Por qué una operación con varios cortes puede seguir teniendo costo amortizado $O(1)$?

Sea una operación de decrease-key que desencadena c cortes en cascada. El costo real es $c_i = O(c)$. El primer corte genera $\Delta\Phi \leq 3$, pero cada uno de los $c - 1$ cortes en cascada subsiguientes desmarca un nodo y crea una raíz, aportando $\Delta\Phi = -1$. El cambio total de potencial es:

$$\Delta\Phi \leq 3 - (c - 1) = 4 - c \quad (5)$$

Sumando el costo real y la variación de potencial:

$$\hat{c}_i = c_i + \Delta\Phi = O(c) - c + 4 = O(1) \quad (6)$$

El potencial previamente almacenado en los nodos marcados absorbe exactamente el costo lineal de la cascada, preservando la cota constante amortizada.

7. ¿Qué errores de implementación detectaron los verificadores de invariantes?

El método `validate()` permitió detectar durante el desarrollo:

- Nodos promovidos a raíz que conservaban erróneamente su referencia a `parent` o mantenían la bandera `mark = True`.
- Desajustes en el grado `degree` de nodos padres al desvincular hijos en cortes simples.
- Asignaciones incorrectas de punteros en la desconexión de listas circulares de un solo elemento (`x.right == x`), que producían ciclos infinitos.

8. ¿Recomendaría esta implementación para un sistema real en Python? Sustente la decisión

****No se recomienda**** para sistemas de producción estándar en Python interpretado. Aunque el montículo de Fibonacci posee una cota asintótica superior para grafos muy densos, la sobrecarga por manejo de objetos dinámicos, referencias de punteros y recolección de basura en CPython introduce constantes multiplicativas tan altas que el módulo `heapq` en lenguaje C resulta más eficiente y confiable en la gran mayoría de las aplicaciones prácticas de la industria.

5. Conclusiones grupales

En la presente práctica de laboratorio se implementó, verificó formalmente y analizó experimentalmente el montículo de Fibonacci y su método de análisis amortizado mediante la función potencial $\Phi(H) = t(H) + 2m(H)$, contrastando su desempeño con el módulo estándar `heapq` de Python en operaciones básicas, secuencias intensivas de disminución de clave y el algoritmo de caminos mínimos de Dijkstra.

La batería exhaustiva de pruebas automáticas certificó el cumplimiento del 100 % de las invariantes estructurales requeridas: orden de montículo, grados exactos por nodo, consistencia bidireccional en listas circulares, ausencia de marcas en las raíces y unicidad de grados tras la consolidación. Se comprobó experimentalmente que el costo amortizado de las operaciones de inserción ($O(1)$), unión ($O(1)$) y disminución de clave ($O(1)$) absorbe rigurosamente el costo de reestructuraciones profundas, tales como cortes en cascada y enlaces múltiples durante la extracción del mínimo ($O(\log n)$), validando la contabilidad teórica donde el potencial liberado financia el trabajo diferido.

En el plano experimental, los resultados evidenciaron la diferencia fundamental entre complejidad asintótica y rendimiento en tiempo de ejecución. En operaciones ordinarias de inserción y extracción completa, la implementación nativa en lenguaje C de `heapq` superó ampliamente al montículo de Fibonacci debido a la contigüidad de memoria y localidad de caché frente a la sobrecarga de punteros de objetos dinámicos en CPython. No obstante, al someter las estructuras a una tasa dominante de disminuciones ($q = 10n$), el montículo de Fibonacci superó a `heapq` (568 ms frente a 767 ms para 200 000 disminuciones), demostrando la ventaja de su cota constante frente a la acumulación desmedida de entradas obsoletas en montículos binarios perezosos (que multiplicaron por once el tamaño físico en memoria). Finalmente, en Dijkstra ambas colas produjeron distancias mínimas idénticas, ratificando que el montículo de Fibonacci es óptimo teóricamente para grafos densos, aunque en sistemas reales interpretados su adopción debe sopesar las elevadas constantes de implementación.